

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 6

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 6

TITLE:

Design and develop a Smart Inventory Management System for Construction Sites. The system tracks materials and inventory movement to improve resource utilization and site

Done By:

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

Project Overview:

The Smart Inventory Management System for Construction Sites is designed to track construction materials and monitor their movement within a construction site. Construction projects use a large number of materials such as cement, steel rods, bricks, pipes, tools, and electrical components.

Manual inventory management can result in material shortages, excess stock, misplaced materials, and inaccurate records. The proposed system uses RFID/Barcode technology, sensors, and an ESP8266/Arduino microcontroller to automate material identification and inventory tracking

. Whenever a material enters or leaves the storage area, its identification is recorded. The inventory information can be transmitted to a cloud platform for remote monitoring and management.

Problem Statement

Construction sites contain large quantities of materials that are continuously received, stored, transferred, and consumed.

Traditional manual inventory management can cause:

- Difficulty tracking materials.
- Incorrect inventory records.
- Material shortages.
- Excess inventory.
- Material loss or misplacement.
- Delayed stock updates.
- Difficulty monitoring material movement.
- Increased labour requirements.
- Poor resource utilization.

Therefore, a smart inventory management system is required to automatically monitor material availability and movement.

.

Proposed Solution

The proposed system uses RFID tags and an RFID reader to identify materials.

Each material or material container is assigned a unique RFID tag.

When the material is brought into the storage area:

Material



RFID Tag



RFID Reader



ESP8266



Inventory Database



Stock Updated

When a material is removed:

Material Removed



RFID Detection



ESP8266



Stock Quantity Reduced



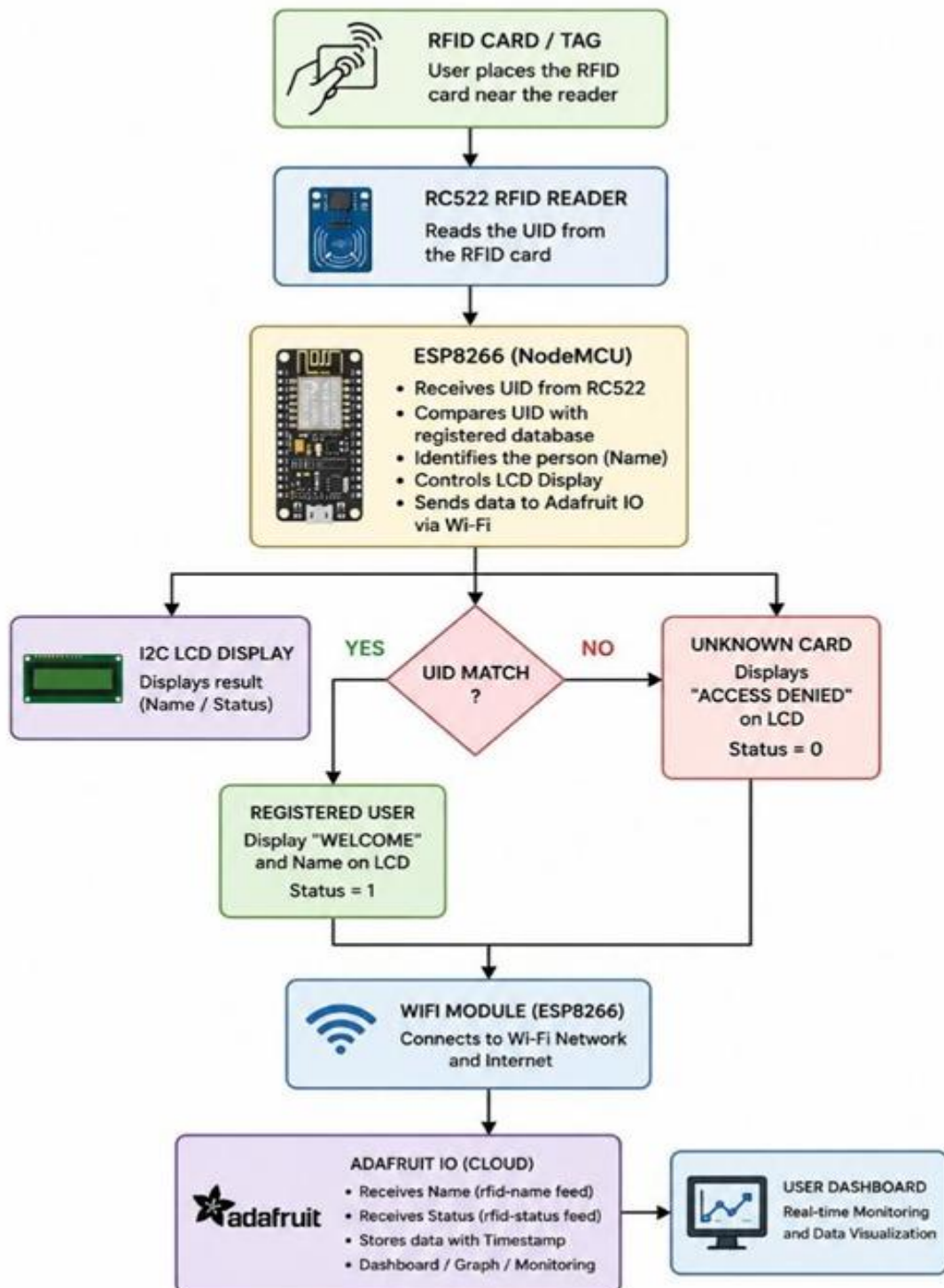
Database Updated



Low Stock Check

System Architecture

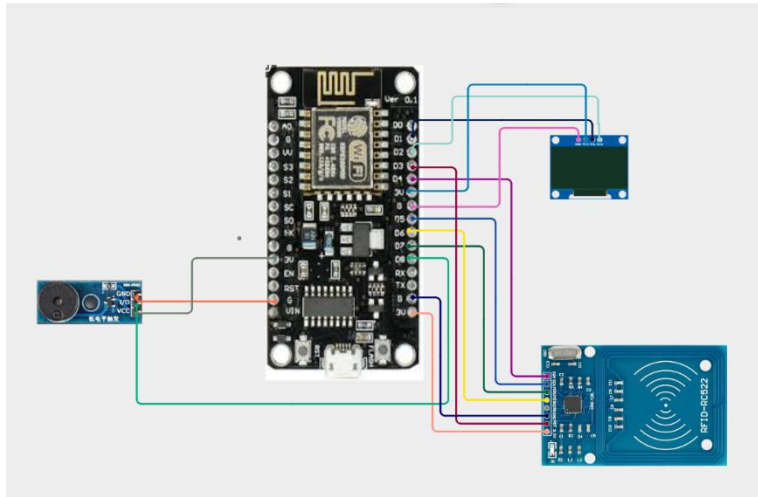
SYSTEM ARCHITECTURE



Circuit Table

S.No.	Component	Pin	ESP8266 Connection
1	MFRC522 RFID	VCC	3.3 V
2	MFRC522 RFID	GND	GND
3	MFRC522 RFID	SDA/SS	D8
4	MFRC522 RFID	SCK	D5
5	MFRC522 RFID	MOSI	D7
6	MFRC522 RFID	MISO	D6
7	MFRC522 RFID	RST	D3
8	OLED Display	VCC	3.3 V
9	OLED Display	GND	GND
10	OLED Display	SDA	D2
11	OLED Display	SCL	D1
12	Green LED	Anode	GPIO through 220 Ω
13	Green LED	Cathode	GND
14	Yellow LED	Anode	GPIO through 220 Ω
15	Yellow LED	Cathode	GND
16	Red LED	Anode	GPIO through 220 Ω
17	Red LED	Cathode	GND
18	Buzzer	Positive	GPIO
19	Buzzer	Negative	GND

Circuit Design



Algorithm

1. START
2. Initialize ESP8266.
3. Initialize RFID reader.
4. Initialize OLED/LCD display.
5. Initialize LEDs and buzzer.
6. Connect ESP8266 to Wi-Fi.
7. Connect to inventory database/cloud.
8. Wait for RFID tag.
9. Detect RFID tag.
10. Read RFID UID.
11. Compare RFID UID with stored

material database.

12. Is the RFID tag registered?

13. If NO:

Display "UNKNOWN MATERIAL".

Activate alert.

Record unauthorized movement.

Return to Step 8.

14. If YES:

Identify material.

15. Determine material movement:

IN or OUT.

16. If IN:

Increase material quantity.

17. If OUT:

Decrease material quantity.

18. Update inventory database.

19. Check current quantity.

20. Is quantity below minimum level?

21. If YES:

Display LOW STOCK.

Activate warning alert.

Send notification.

22. If NO:

Display AVAILABLE.

Turn ON green indicator.

23. Record date and time.

24. Return to RFID scanning.

25. Repeat continuously.

26. STOP.

Coding

```
// =====  
  
// CONSTRUCTION WORKER HEALTH & SAFETY MONITORING  
SYSTEM  
  
// ESP8266 NodeMCU (GPIO Version)  
  
// GPS + Pulse + MQ2 + DHT11 + Adafruit IO  
  
// Upload every 20 seconds (3 updates/minute)  
  
// Active LOW Buzzer  
  
// =====  
  
#include <ESP8266WiFi.h>  
  
#include "AdafruitIO_WiFi.h"  
  
#include <TinyGPSPlus.h>  
  
#include <SoftwareSerial.h>  
  
#include <DHT.h>  
  
  
// ----- WiFi -----  
  
#define WIFI_SSID "Harish Kumar K K"  
  
#define WIFI_PASS "Mavboiii@10"
```

```
// ----- Adafruit IO -----
```

```
#define IO_USERNAME "Jr_Frost"
```

```
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

```
// ----- Feeds -----
```

```
AdafruitIO_Feed *tempFeed    = io.feed("temperature");
```

```
AdafruitIO_Feed *humFeed     = io.feed("humidity");
```

```
AdafruitIO_Feed *pulseFeed   = io.feed("heart-rate");
```

```
AdafruitIO_Feed *gasFeed     = io.feed("gas-status");
```

```
AdafruitIO_Feed *alertFeed   = io.feed("safety-alert");
```

```
AdafruitIO_Feed *latitudeFeed = io.feed("latitude");
```

```
AdafruitIO_Feed *longitudeFeed = io.feed("longitude");
```

```
AdafruitIO_Feed *locationFeed = io.feed("gps-location");
```

```
// ----- GPIO Pins -----
```

```
#define PULSE_PIN  A0
```

```
#define GAS_PIN    5    // GPIO5
```

```
#define DHT_PIN    4    // GPIO4
```

```
#define BUZZER_PIN 13    // GPIO13
```

```
#define DHTTYPE DHT11
```

```
// GPS
```

```
SoftwareSerial gpsSerial(14, 12); // RX=GPIO14, TX=GPIO12
```

```
TinyGPSPlus gps;
```

```
DHT dht(DHT_PIN, DHTTYPE);
```

```
// Upload every 20 seconds
```

```
const unsigned long uploadInterval = 20000;
```

```
unsigned long lastUpload = 0;
```

```
// Pulse Variables
```

```
int BPM = 0;
```

```
bool beatDetected = false;
```

```
const int threshold = 550;
```

```
unsigned long lastBeatTime = 0;
```

```
void readPulse() {
```

```
    int signal = analogRead(PULSE_PIN);
```

```
if (signal > threshold && !beatDetected) {
```

```
    beatDetected = true;
```

```
    unsigned long now = millis();
```

```
    if (lastBeatTime != 0)
```

```
        BPM = 60000 / (now - lastBeatTime);
```

```
    lastBeatTime = now;
```

```
}
```

```
if (signal < threshold - 40)
```

```
    beatDetected = false;
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    gpsSerial.begin(9600);
```

```
dht.begin();
```

```
pinMode(GAS_PIN, INPUT);
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
// Active LOW Buzzer
```

```
digitalWrite(BUZZER_PIN, HIGH);
```

```
Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println(" WORKER HEALTH & SAFETY SYSTEM ");
```

```
Serial.println("=====");
```

```
Serial.print("Connecting to Adafruit IO");
```

```
io.connect();
```

```
while (io.status() < AIO_CONNECTED) {
```

```
    Serial.print(".");
```

```
    delay(500);
```

```
}
```

```
Serial.println("\nConnected Successfully!");

Serial.println("Waiting for GPS Fix...");

}

// =====

// Main Loop

// =====

void loop() {

    io.run();

    readPulse();

    while (gpsSerial.available()) {
        gps.encode(gpsSerial.read());
    }

    float temperature = dht.readTemperature();

    float humidity = dht.readHumidity();

    bool gasDetected = (digitalRead(GAS_PIN) == LOW);
```

```
bool danger = false;
```

```
String alertMessage = "";
```

```
if (!isnan(temperature) && temperature > 40) {
```

```
    danger = true;
```

```
    alertMessage += "HIGH TEMP ";
```

```
}
```

```
if (gasDetected) {
```

```
    danger = true;
```

```
    alertMessage += "GAS LEAK ";
```

```
}
```

```
if (BPM > 120 || (BPM > 0 && BPM < 50)) {
```

```
    danger = true;
```

```
    alertMessage += "ABNORMAL PULSE ";
```

```
}
```

```
// Active LOW Buzzer
```

```
if (danger)
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

else

digitalWrite(BUZZER_PIN, HIGH);

if (millis() - lastUpload >= uploadInterval) {

lastUpload = millis();

Serial.println("-----");

Serial.print("Temperature : ");

Serial.print(temperature);

Serial.println(" C");

Serial.print("Humidity : ");

Serial.print(humidity);

Serial.println(" %");

Serial.print("Heart Rate : ");

Serial.print(BPM);

Serial.println(" BPM");

Serial.print("Gas Status : ");


```
Serial.println(gasDetected ? "DETECTED" : "SAFE");
```

```
// ----- THROTTLE SAFE UPLOADS -----
```

```
tempFeed->save(temperature);
```

```
delay(1200);
```

```
humFeed->save(humidity);
```

```
delay(1200);
```

```
pulseFeed->save(BPM);
```

```
delay(1200);
```

```
gasFeed->save(gasDetected ? 1 : 0);
```

```
delay(1200);
```

```
alertFeed->save(danger ? alertMessage : "SAFE");
```

```
delay(1200);
```

```
// GPS Upload
```

```
if (gps.location.isValid()) {
```

```
double latitude = gps.location.lat();  
double longitude = gps.location.lng();  
double altitude = gps.altitude.isValid() ? gps.altitude.meters() : 0;
```

```
Serial.print("Latitude : ");  
Serial.println(latitude, 6);
```

```
Serial.print("Longitude : ");  
Serial.println(longitude, 6);
```

```
latitudeFeed->save(latitude);  
delay(1200);
```

```
longitudeFeed->save(longitude);  
delay(1200);
```

```
locationFeed->save(0, latitude, longitude, altitude);  
delay(1200);
```

```
Serial.println("GPS Uploaded Successfully.");
```

```
} else {  
  
    Serial.println("Waiting for GPS satellites...");  
  
}  
  
if (danger) {  
    Serial.print("ALERT : ");  
    Serial.println(alertMessage);  
} else {  
    Serial.println("Worker Status : SAFE");  
}  
  
Serial.println("-----");  
}  
}
```

System Working

Step 1 – Material Tagging

Each construction material or material container is assigned an RFID tag.

Example:

RFID TAG → Cement → MAT001

Step 2 – Material Identification

When the material passes near the RFID reader, the reader detects its unique ID.

Step 3 – Data Processing

The ESP8266 compares the RFID ID with the stored inventory database.

Step 4 – Stock Update

When material enters the storage area:

Old Stock + Received Quantity

↓

New Stock

When material is taken from the storage area:

Old Stock - Used Quantity

↓

New Stock

Step 5 – Low Stock Detection

The controller compares the current quantity with the minimum stock level.

Example:

Cement Stock = 15 bags

Minimum Stock = 20 bags

$15 < 20$

↓

LOW STOCK ALERT

Step 6 – Cloud Monitoring

Inventory information can be transmitted to a cloud database through Wi-Fi. The supervisor can monitor the stock remotely.

Bills Of Material

S.N o.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU	Wi-Fi Microcontroller	1	Main Controller
2	MFRC522 RFID Reader	13.56 MHz	1	RFID Detection
3	RFID Tags	13.56 MHz	5+	Material Identification
4	OLED Display	0.96-inch I2C	1	Inventory Display
5	Green LED	5 mm	1	Available Status
6	Yellow LED	5 mm	1	Low Stock
7	Red LED	5 mm	1	Alert
8	Resistor	220 Ω	3	LED Protection
9	Buzzer	3.3/5 V	1	Alert
10	Breadboard	Standard	1	Prototype Assembly
11	Jumper Wires	Male-Male	1 Set	Circuit Connections
12	USB Cable	Micro USB	1	Programming/Power
13	Power Supply	Suitable regulated supply	1	System Power

Prototype Implementation

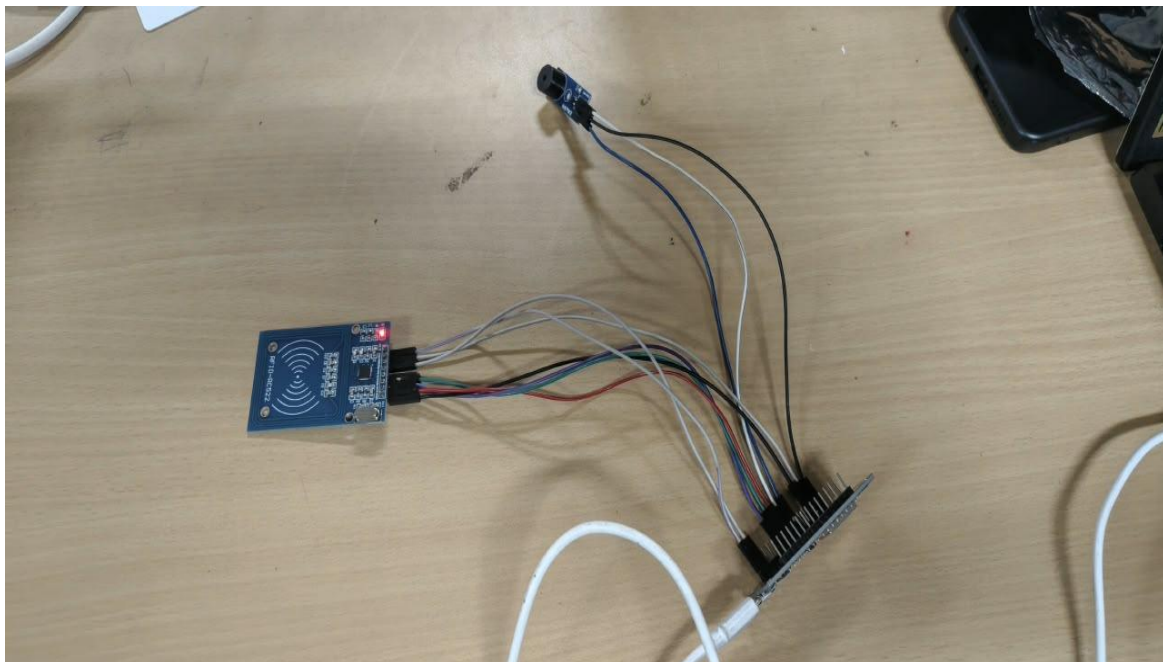
The prototype can be implemented at a construction-site material storage area. Each material container is provided with an RFID tag. The RFID reader is installed near the storage entrance or material issue point.

When a material enters the storage area, the RFID tag is scanned and the inventory quantity is increased.

When material is removed, the tag is scanned again and the quantity is reduced.

The ESP8266 updates the local display and can send the transaction to a cloud database.

A supervisor can use the dashboard to monitor the current stock and identify materials that require replenishment.



Results & Performance Analysis

The system successfully demonstrates automated identification and tracking of construction materials.

Parameter	Result
RFID Material Identification	Yes
Material Entry Tracking	Yes
Material Exit Tracking	Yes
Stock Quantity Monitoring	Yes
Low Stock Detection	Yes
Unknown Tag Detection	Yes
Audible Alert	Yes
Visual Indication	Yes
Cloud Integration	Supported
Remote Monitoring	Supported

Performance Analysis

The system reduces manual inventory recording and provides faster access to material information.

It can help construction-site managers:

- Monitor available stock.
- Identify material movement.
- Reduce material loss.
- Detect low-stock conditions.
- Plan material purchases.
- Improve resource utilization.
- Maintain transaction history.

Jr_Frost / Dashboards / Smart Inventory Management System

+ New Block

Edit Layout



AF91D648

Jr_Frost / Dashboards / Smart Inventory Management System

+ New Block

Edit Layout



C1F65968

Created at	Value	Location	
2026/08/19 10:58:52AM	AF91D648	0.0, 0.0, 0.0	×
2026/08/19 10:58:23AM	6FCBEB48	0.0, 0.0, 0.0	×
2026/08/19 10:58:08AM	1F22EB48	0.0, 0.0, 0.0	×
2026/08/19 10:57:54AM	6F22CB48	0.0, 0.0, 0.0	×
2026/08/19 10:57:37AM	CFFFDB48	0.0, 0.0, 0.0	×
2026/08/19 10:55:01AM	BA8A8132	0.0, 0.0, 0.0	×
2026/08/19 10:54:47AM	C1F65968	0.0, 0.0, 0.0	×
2026/08/19 10:54:34AM	7BABD13E	0.0, 0.0, 0.0	×
2026/08/19 10:53:56AM	C1F65968	0.0, 0.0, 0.0	×
2026/08/19 10:53:42AM	BA8A8132	0.0, 0.0, 0.0	×

```
...
Adafruit IO Connected
Scan RFID Tag...
-----
Material Tag UID: BA8A8132
Inventory Updated Successfully
-----
Material Tag UID: C1F65968
Inventory Updated Successfully
-----
Material Tag UID: 7BABD13E
Inventory Updated Successfully
-----
Material Tag UID: C1F65968
Inventory Updated Successfully
-----
Material Tag UID: BA8A8132
Inventory Updated Successfully
-----
Material Tag UID: CFFFDB48
Inventory Updated Successfully
-----
Material Tag UID: 6F22CB48
Inventory Updated Successfully
-----
Material Tag UID: 1F22EB48
Inventory Updated Successfully
-----
Material Tag UID: 6FCBEB48
Inventory Updated Successfully
-----
Material Tag UID: AF91D648
Inventory Updated Successfully
-----
```